The $0 Content Engine: Complete Blueprint

Every script, prompt, command, and workflow behind a fully automated short-form content business — built and run in production by a one-person company

By Already Here LLC — the team behind @alreadyherellc

What you bought, and what it does

This is not a book of ideas. It is the complete build manual for the system that publishes our YouTube Shorts, Instagram Reels, and Facebook videos every week while we sleep — trend research, scriptwriting, voiceover, captions, B-roll, posting, reposting, and the weekly newsletter, end to end, for $0 in tooling.

The free edition of *The $0 Content Engine* told you what the five stages are. This blueprint gives you everything the free edition deliberately left out:

- The exact trend-mining code and the scoring formula, ready to paste
- All 5 script format templates with fill-in-the-blank hooks
- The complete LLM prompt chain, including the free-tier fallback order
- The full render pipeline: every ffmpeg filter, every edge-tts flag
- The publishing setup for all three platforms, including the two token gotchas that cost us days
- The complete GitHub Actions workflow files, cron schedules included
- The repost engine rules and the newsletter composer
- A 30-day launch calendar and a troubleshooting table for every failure we've actually hit

Work through it in order the first time. After that it's a reference manual.

Prerequisites: a computer with Python 3.10+, a free GitHub account, and about one weekend. No design skills, no camera, no editing software, no subscriptions.

Part 1 — Mine what's already winning

Guessing topics is how channels die. YouTube search results are a free market-research database if you rank them correctly.

The metric: views per day

Total views lie. A 3-year-old video with 2M views earns ~1,800 views/day — a proven evergreen topic. A 5-day-old video with 40k views earns 8,000/day — a rising trend. Rank by **views ÷ age in days** and both float to the top; stale one-hit wonders sink.

The tool: yt-dlp, metadata only

`yt-dlp` reads search results without downloading anything and without an API key.

```
pip install yt-dlp
```

The two-phase pattern that keeps it fast:

- **Flat search** — pull the top 25 results per niche query with `extract_flat: True`. This is one cheap request per query and returns title, views, and duration.
- **Detail fetch** — for only the top 3 by raw views, fetch the upload date and compute views/day.

```python
import yt_dlp

from datetime import datetime, timezone

def flat_search(query, depth=25, min_s=12, max_s=150):

opts = {"quiet": True, "extract_flat": True, "skip_download": True}

with yt_dlp.YoutubeDL(opts) as ydl:

info = ydl.extract_info(f"ytsearch{depth}:{query}", download=False)

out = []

for e in (info or {}).get("entries") or []:

views, dur = e.get("view_count"), e.get("duration")

if not views or not dur or not (min_s <= dur <= max_s):
```

continue # not a Short, or too new to have data

out.append({"id": e["id"], "title": e.get("title", ""),

"views": views, "duration": dur,

"url": f"https://www.youtube.com/watch?v={e['id']}"})

return sorted(out, key=lambda x: x["views"], reverse=True)

def views_per_day(video):

opts = {"quiet": True, "skip_download": True}

with yt_dlp.YoutubeDL(opts) as ydl:

info = ydl.extract_info(video["url"], download=False)

up = info.get("upload_date") # YYYYMMDD

if up:

age = max((datetime.now(timezone.utc)

- datetime.strptime(up, "%Y%m%d")

.replace(tzinfo=timezone.utc)).days, 1)

video["views_per_day"] = round(video["views"] / age)

return video

```

The duration window (12–150 seconds) filters out long-form results so you're only studying Shorts.

*Choosing your niche queries*

Write 10–15 search queries, not 1. Ours mix three buckets — copy the structure, swap the subjects:

| Bucket | Purpose | Query pattern |

|---|---|---|

| Money queries | Topics tied to products you can monetize | `{product category} tutorial shorts` |
```

| Format queries | Proven emotional formats | `{topic} secrets shorts`, `make money {topic} story shorts` |

| Volume queries | Broad niches for reach | `ai tools you need shorts`, `productivity tips shorts` |

Run the miner on a schedule and store results as JSON. Every downstream stage reads from that file — that's what makes the pipeline composable.

The compliance rule that keeps your channel alive

Mine the topic. Never touch the video. Re-uploading or clipping someone else's video earns copyright strikes and reuse-policy demonetization. Recreating the *topic* with your own script and licensed footage is simply publishing. This one rule is why this system survives when "faceless channel" tutorials get accounts banned.

Worksheet: your first mining run

- List 3 products/affiliate programs you could plausibly promote
- Write 3 money queries, 5 format queries, 4 volume queries
- Run the miner; keep every topic scoring above 500 views/day
- Delete anything you couldn't write 3 honest tips about
- The top 5 that survive are your week-one videos

Part 2 — Scripts that write themselves

A Short's script is 55 words. That is not a style choice — 55 spoken words ≈ 20 seconds, the retention sweet spot where completion rate (the metric the algorithm actually rewards) stays high.

The two-layer factory

Layer 1 — free LLM chain. Several providers have genuinely free tiers. Chain them in order and fall through on any error:

- Gemini (free API tier)
- Groq (free tier, very fast)
- OpenRouter free models

Each gets the same prompt. If all three fail — rate limits, outages, no keys — you fall to Layer 2 and the pipeline still ships.

Layer 2 — the template engine. A library of fill-in-the-blank structures assembled from the topic's benefit list. Templates never rate-limit, never hallucinate, and never expire. Roughly a third of our published videos are pure template output; nobody can tell.

The master prompt (Layer 1)

`

Write a spoken script for a YouTube Short about: {topic}

Rules:

- Exactly 50-60 words, spoken English, no stage directions
- Structure: HOOK (one arresting sentence) -> SUBSTANCE

(2-3 concrete, specific tips or facts) -> CTA (one short sentence)

- The hook must create an information gap or challenge an assumption
- Concrete numbers beat adjectives. "Cut render time to 60 seconds"

not "much faster"

- No hashtags, no emojis, no "in this video"
- Reading level: 7th grade

Format: {format_name} - {format_instruction}

`

The 5 formats and when to use each

Matching format to topic is worth more than any single "viral hook."

1. TIPS — numbered list. For how-to and tool topics.

> Hook: "Three {topic} tricks nobody shows beginners." → tip 1 → tip 2 → tip 3 → "Follow for more."

2. SECRETS — insider tone. For industry/money topics.

> Hook: "Nobody tells you this about {topic}." → the insider fact → why it's hidden → what to do instead → CTA.

3. STORY — struggle → discovery → payoff. For make-money and transformation topics.

> Hook: "I wasted six months on {topic} until this." → the struggle (1 sentence) → the discovery → the result with a number → CTA.

4. DIY — three steps. For build/setup topics.

> Hook: "Build {thing} in three steps, zero budget." → step 1 → step 2 → step 3 → "Save this."

5. SALES — problem → agitation → solution. Use at most 1 in 5 videos.

> Hook: "{Pain point} is costing you {number}." → agitate → the solution → where to get it.

Evergreen hook bank (template Layer 2 fills from these)

- "Nobody tells you this about ___"
- "Stop doing ___ — do this instead"
- "The ___ mistake that costs beginners ___"
- "I tested ___ so you don't have to"
- "Three signs your ___ is broken"
- "___ in 20 seconds"

Quality gate before anything renders

- [ ] 45–65 words when read aloud
- [ ] Hook contains no throat-clearing ("Hey guys, today…" = delete)
- [ ] At least one concrete number or named tool
- [ ] CTA is 6 words or fewer
- [ ] You would not be embarrassed to voice this yourself

Part 3 — Studio-grade rendering, free

Four ingredients, all free, assembled by one script.

Voiceover: edge-tts

Microsoft Edge's neural voices via the `edge-tts` Python package — no account, no key, no watermark, and crucially it returns **word-level timestamps** as it synthesizes.

```
pip install edge-tts
```

```python

import edge_tts, asyncio, json

async def speak(text, out_mp3="vo.mp3", voice="en-US-ChristopherNeural"):

words = []

tts = edge_tts.Communicate(text, voice, rate="+8%")

with open(out_mp3, "wb") as f:

async for chunk in tts.stream():

if chunk["type"] == "audio":

f.write(chunk["data"])

elif chunk["type"] == "WordBoundary":

words.append({"word": chunk["text"],

"start": chunk["offset"] / 1e7,

"end": (chunk["offset"] + chunk["duration"]) / 1e7})

return words

```

`rate="+8%"` matters: slightly faster than default reads as energetic instead of robotic. Try `en-US-ChristopherNeural` (male) and `en-US-JennyNeural` (female); pick one and stay consistent — voice is brand.

Captions: the "CapCut look" from those timestamps

Word-by-word pop captions are the single highest-impact retention feature, and the word timestamps make them free. Generate an ASS subtitle file where each word gets its own event, styled big/bold/center:

```

[V4+ Styles]

Style: Word,Arial Black,120,&H00FFFFFF,&H000000FF,&H00000000,&H96000000,-1,0,0,0,100,100,0,0,1,8,4,5,60,6

[Events]
```

Dialogue: 0,0:00:00.05,0:00:00.38,Word,,0,0,0,,
{\fscx70\fscy70\t(0,80,\fscx100\fscy100)}NOBODY

Dialogue: 0,0:00:00.38,0:00:00.71,Word,,0,0,0,,
{\fscx70\fscy70\t(0,80,\fscx100\fscy100)}TELLS

```

The `\t(0,80,\fscx100\fscy100)` transform is the pop-in scale animation. Write one Dialogue line per word from the timestamp list, uppercase, and ffmpeg burns them in.

*B-roll: Pexels*

Pexels' API is free and every clip is licensed for commercial use, no attribution required (check the license page for the current terms — they've been stable for years).

- Register at pexels.com/api for a key
- Search `https://api.pexels.com/videos/search?query={keyword}&orientation=portrait&per_page=5`
- Pull 3–4 clips per video, matched to keywords extracted from the script
- Cache downloads by keyword so repeat topics cost zero bandwidth

*Assembly: one ffmpeg command chain*

Per video: trim each clip to 3–5s, scale/crop to 1080×1920, crossfade between clips, burn captions, duck music under the voice, add a swoosh on each transition.

The core filter graph (three clips shown):

```

ffmpeg -i clip1.mp4 -i clip2.mp4 -i clip3.mp4 -i vo.mp3 -i music.mp3 \

-filter_complex "\

[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,setsar=1[v0];\

[1:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,setsar=1[v1];\

[2:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,setsar=1[v2];\

[v0][v1]xfade=transition=fade:duration=0.3:offset=3.7[x1];\

[x1][v2]xfade=transition=fade:duration=0.3:offset=7.4[xv];\

[xv]ass=captions.ass[outv];\
```
```

[4:a]volume=0.12[m];[3:a][m]amix=inputs=2:duration=first[outa]" \

-map "[outv]" -map "[outa]" -t 20 -r 30 -c:v libx264 -preset fast \

-pix_fmt yuv420p out.mp4

`

Music at `volume=0.12` under full-volume voice is the mix that sounds professional. Free music: YouTube's Audio Library (safe for all platforms) — download a handful, rotate them.

Render time on an ordinary laptop: about a minute per video.

Render checklist

- [ ] 1080×1920, 30fps, H.264, yuv420p (Instagram rejects odd pixel formats)
- [ ] Duration = voiceover length + 0.5s tail, never over 60s
- [ ] Captions inside the center-safe area (platform UI covers top and bottom ~250px)
- [ ] Audio peaks around −3dB; music never fights the voice
- [ ] Watch one full render per batch before publishing — 20 seconds of QA

Part 4 — Publish everywhere, automatically

YouTube Shorts: Data API

One-time setup (~20 minutes):

- console.cloud.google.com → create a project → enable **YouTube Data API v3**
- OAuth consent screen → External → add your own Google account as a test user
- Credentials → Create OAuth client ID → **Desktop app** → note client ID + secret
- Run a one-time local script to authorize scope `https://www.googleapis.com/auth/youtube` and capture the **refresh token** — that's the credential that lasts
- Store all three as environment secrets: `YOUTUBE_CLIENT_ID`, `YOUTUBE_CLIENT_SECRET`, `YOUTUBE_REFRESH_TOKEN`

Upload with `google-api-python-client`, category 27 (Education) or 28 (Science & Tech), and put your links in the description (see Part 7). Include `#shorts` in the title or description. Unverified API projects cap at ~6 uploads/day — far more than you need.

Gotcha #1: request the full `youtube` scope from the start, not just `youtube.upload`. The upload-only scope can't edit descriptions later, and you *will* want to retro-fix descriptions. We had to re-mint our token to fix six live videos.

Instagram Reels + Facebook: Graph API

You need: an Instagram **professional** account, a Facebook Page, both linked, and a Meta developer app.

- developers.facebook.com → create an app → add Instagram Graph API + Facebook Login products
- Add your Instagram account as an **Instagram Tester** under App roles → Roles (the invite must be *accepted* inside Instagram: Settings → Website permissions → Tester invites)
- Graph API Explorer → generate a token with `instagram_basic`, `instagram_content_publish`, `pages_show_list`, `pages_manage_posts`
- Exchange for a long-lived token (60 days) and store as secrets

Reels publishing is a two-step dance: create a media container with a **public video URL**, poll until `status_code=FINISHED`, then publish:

`

POST graph.instagram.com/{ig-user-id}/media

?media_type=REELS&video_url={public_url}&caption={caption}

GET graph.instagram.com/{container-id}?fields=status_code (poll ~every 5s)

POST graph.instagram.com/{ig-user-id}/media_publish?creation_id={container-id}

`

Gotcha #2 (the one nobody documents): Instagram now issues two token generations. Newer tokens (they start with `IG`) only work against **graph.instagram.com**; older Page-based tokens only work against **graph.facebook.com**. Route by token prefix and both work. Point a mixed integration at one host and half your calls 400 with misleading errors.

Facebook Page video posting is one call — `POST graph.facebook.com/{page-id}/videos` with `file_url` — using a **Page access token**, which (via the token-debugger "never expires" variety) outlives every other Meta credential. Set it once.

Hosting the video file: GitHub Releases

Instagram fetches your video from a URL, so you need free, reliable public hosting: attach the MP4s to a **GitHub release**. Release assets have stable direct-download URLs, no bandwidth bill, and the `gh` CLI uploads them in one line:

```
gh release create shorts-2026-07-12 out1.mp4 out2.mp4 --title "weekly batch"
```

Fail-soft is the architecture

Wrap every platform call so one failure never kills the batch:

```python
results = {"youtube": try_youtube(v), "instagram": try_instagram(v),
"facebook": try_facebook(v)}
# log failures, publish successes, exit 0 either way
```

An expired Instagram token should cost you Instagram for a day — not the whole week's output. This single pattern is the difference between a system that runs for months and one you babysit.

Part 5 — Scheduling: GitHub Actions runs the company

GitHub Actions gives private repos 2,000 free minutes/month; our entire weekly pipeline uses under 60.

The complete weekly workflow:

```yaml
name: weekly-content
on:
  schedule:
    - cron: "0 2 0" # Sundays 02:00 UTC — posts before you wake
```

```
workflow_dispatch: {} # manual run button, always include this

jobs:

produce-and-post:

runs-on: ubuntu-latest

timeout-minutes: 45

steps:

- uses: actions/checkout@v4

- uses: actions/setup-python@v5

with: { python-version: "3.12" }

- run: sudo apt-get update && sudo apt-get install -y ffmpeg

- run: pip install yt-dlp edge-tts requests google-api-python-client google-auth

- name: mine trends

run: python scripts/find_trending.py

- name: render videos

run: python scripts/generate_shorts.py --limit 5

- name: host on release

env: { GH_TOKEN: "${{ github.token }}" }

run: gh release create "shorts-$(date +%F)" out/*.mp4 --title "batch $(date +%F)"

- name: publish everywhere

env:

YOUTUBE_CLIENT_ID: ${{ secrets.YOUTUBE_CLIENT_ID }}

YOUTUBE_CLIENT_SECRET: ${{ secrets.YOUTUBE_CLIENT_SECRET }}

YOUTUBE_REFRESH_TOKEN: ${{ secrets.YOUTUBE_REFRESH_TOKEN }}

IG_USER_ID: ${{ secrets.IG_USER_ID }}

IG_ACCESS_TOKEN: ${{ secrets.IG_ACCESS_TOKEN }}
```

FB_PAGE_ID: ${{ secrets.FB_PAGE_ID }}

FB_PAGE_ACCESS_TOKEN: ${{ secrets.FB_PAGE_ACCESS_TOKEN }}

run: python scripts/post_social.py

```

Our production cadence, copy it or shift it:

| Job | Cron | What it does |

|---|---|---|

| Weekly content | `0 2 0` | mine → render → host → post |

| Repost cycle | `0 15 3` | repost best performer 2+ weeks old |

| Newsletter | `0 13 1` | compose + publish Monday issue |

| Revenue report | `30 12 *` | daily numbers into the repo |

Rules that keep Actions healthy: always add `workflow_dispatch` (you'll thank yourself when debugging), always set `timeout-minutes`, never print secrets, and let every posting step fail soft.

---

## Part 6 — The recycle loop and the newsletter

*Reposting: your back catalog is free inventory*

Once a video is 2+ weeks old its performance is knowable. Weekly job:

- Pull your uploads and view counts (one YouTube API call)
- Filter: older than 14 days, not already a repost, above-median views
- Repost the winner to IG + FB with a fresh caption prefix: "ICYMI:", "Still true:", "Most-watched this month:"
- **Mark it** in a JSON ledger so you never repost a repost

One good video becomes 4–6 impressions cycles over a year. This is the highest ROI script in the whole system — it creates output with zero new production.

*The Monday newsletter*

Every Monday, a script assembles: this week's published videos (from the upload ledger), the top 3 trending topics (from the miner — "what the data says is working"), and a
```

standing CTA block. Delivery fails soft: email platform API → static page on GitHub Pages → local HTML artifact. The newsletter is also where warm audiences buy — every issue links your products.

Fifteen minutes of setup, runs forever, compounds an owned audience — the only audience an algorithm can't take away.

Part 7 — The money layer

Traffic without an offer is a hobby. Stack these four, in this order:

1. Affiliate links (day one)

Join programs adjacent to your niches. Put links in **every** video description, injected automatically from a JSON map so adding a program updates all future videos:

```
AFFILIATE_LINKS = {"seo-tools": "https://...", "hosting": "https://..."}
```

Description template every video gets:

```
→ Try it: {affiliate link for this video's program}

→ Free playbook — The $0 Content Engine: {lead magnet URL}

→ Our tools & kits: {store URL}
```

Compliance is not optional: every piece of content with an affiliate link carries an FTC disclosure line. Bake it into the template so it can't be forgotten.

2. The free lead magnet (week one)

A genuinely useful free guide (the free edition of this book is ours), hosted for $0 on GitHub Pages, linked from every description. Its job: convert anonymous viewers into email subscribers and warm buyers.

3. Your paid product (week two — this book is ours)

Package your deepest expertise as a $19–39 digital product on Gumroad or Payhip. Free to set up; they take a cut only when you get paid. The formula this book follows, so you can copy it:

- Free edition = the *what* and the *why* → builds trust, ranks, gets shared
- Paid edition = the *exactly how* → every template, command, and gotcha
- Every video description, newsletter issue, and the free edition itself links to the paid edition

4. Services (when inbound starts)

Once the system visibly works, "build this for my business" emails arrive. A done-for-you setup of this exact pipeline is a $1,500–5,000 engagement. The content engine is its own portfolio.

The honest math

At $30 with ~10% platform fees you net ~$27. Ten sales/day ≈ $8,100/month net. Getting there is a traffic problem, not a product problem: at a 2% landing-page conversion you need ~500 targeted visitors/day. Automated posting across 3–4 platforms plus reposts plus a newsletter is how you build toward that without buying ads — but expect the curve to take months, not weeks. Everyone selling faster timelines is selling you a timeline.

Part 8 — The 30-day launch calendar

Week 1 — Foundation. Install Python, ffmpeg, yt-dlp, edge-tts. Write your niche queries, run the miner. Set up YouTube OAuth. Render your first video by hand and study it. Publish 2 videos.

Week 2 — Automation. Wire the full render script. Set up the Meta app, tester invites, tokens. Stand up the GitHub Actions weekly workflow. Publish 3–4 videos, at least one fully hands-free.

Week 3 — Funnel. Write your free lead magnet (one afternoon — it's the *what*, not the *how*). Publish it on GitHub Pages. Add the three-line link block to every description. Start the Monday newsletter. Join 2 affiliate programs.

Week 4 — Money. Outline your paid product from questions your audience actually asks. Set up the store and sales page. Launch the repost cycle. Review 30 days of data: kill the worst-performing niche query, double the best.

The only rule: post consistently for 30 days before judging anything. Short-form data is noisy; the algorithm tests you before it feeds you.

Part 9 — Troubleshooting: every failure we've actually hit

| Symptom | Cause | Fix |

|---|---|---|

| IG container stuck `IN_PROGRESS` forever | Video URL not publicly fetchable | Host on a GitHub release; test the URL in an incognito window |

| IG API 400 with vague OAuth error | Token generation vs. host mismatch | `IG…` tokens → graph.instagram.com; Page tokens → graph.facebook.com |

| YouTube upload 403 | OAuth scope too narrow or consent screen unverified | Full `youtube` scope; add yourself as test user |

| Can't edit old video descriptions | Token minted with upload-only scope | Re-mint refresh token with full `youtube` scope |

| edge-tts suddenly silent/erroring | Transient service change | `pip install -U edge-tts` — fixes it every time |

| ffmpeg "height not divisible by 2" | Odd crop dimensions | Force even: `crop=1080:1920`, `setsar=1` |

| Captions off-screen on IG | Platform UI overlay | Keep text inside center 60% vertically |

| Actions job dies at 6h | No timeout, hung API poll | `timeout-minutes: 45` + bounded poll loops |

| Reposted a repost | No ledger | Mark reposts in JSON; filter on the marker |

| One platform down kills the batch | Exceptions bubbling up | Fail-soft wrapper per platform, always exit 0 |

| Copyright claim | Someone reused source footage | Only Pexels/licensed B-roll — never "found" clips |

Appendix — The complete stack, itemized

| Function | Tool | Cost |

|---|---|---|

| Trend data | yt-dlp | $0 |

| Scriptwriting | Free-tier LLMs + template engine | $0 |

| Voiceover | edge-tts | $0 |

| Captions | Word timestamps + ASS + ffmpeg | $0 |

| B-roll | Pexels API | $0 |

| Music | YouTube Audio Library | $0 |

| Assembly | ffmpeg | $0 |

| YouTube posting | Data API v3 | $0 |

| IG/FB posting | Graph API | $0 |

| Video hosting | GitHub Releases | $0 |

| Scheduling | GitHub Actions | $0 |

| Lead magnet + sales page hosting | GitHub Pages | $0 |

| Store/checkout | Gumroad / Payhip | % of sales only |

| Newsletter | Beehiiv free tier / static page | $0 |

Total fixed cost: $0/month. The only fees you ever pay are a percentage of money you've already made.

Questions, stuck on a step, or want this built for your business? Email dispatch@alreadyherellc.com — a human answers. Follow the build in public at @alreadyherellc.

© 2026 Already Here LLC. Licensed for the personal use of the purchaser. Platform policies change — verify current terms. Affiliate links in our content may earn us a commission at no cost to you.